

Serving Frontier MoE Models on Blackwell: A Complete Syllabus

Target competence: Extract near-roofline performance from B200/B300 nodes and NVL72 racks serving 700B–2.8T parameter sparse MoE models, across decode throughput, TTFT, ITL, and concurrent-user goodput.

Assumed starting point: Comfortable CUDA C++ on consumer Ampere (SM86). No exposure to SM90 (Hopper), SM100/SM103 (Blackwell), warp specialisation, TMA, or any multi-GPU communication primitive.

0. How to use this document

Each module lists **topics** → **subtopics**, a **deliverable** (the artifact that proves you learned it, not just read it), and **primary sources** (go to these, not to blog posts, when the AI tutor and reality disagree).

The modules are ordered by dependency, not by interest. Module 2 is unintelligible without Module 1. Module 11 is unintelligible without Modules 9 and 10.

The tutor prompt is at the end. Use it per subtopic, not per module — a module is a week, a subtopic is a session.

A candid framing before you begin

Four things worth internalising, because they determine whether this syllabus pays off:

1. Your CUDA experience transfers less than you'd hope. SM86 programming — threads, warps, shared memory, coalescing, occupancy — is roughly 30% of what matters on SM100. Blackwell's tensor core path is issued by a *single elected thread*, accumulates into a memory space (TMEM) that doesn't exist on your 3050, and moves data via a DMA engine (TMA) you've never programmed. You are not upgrading a skill; you are

learning an adjacent one that shares vocabulary. Budget accordingly and don't let familiarity with the words create false confidence.

2. The binding constraint is hardware access, not comprehension. You cannot learn any of Modules 9–14 by reading. A single B200 rents for roughly \$6–9/hr on neoclouds; an 8×B200 node for \$30–50/hr; NVL72 slices are effectively unobtainable without a contract. Every hour you spend on a rented node without a pre-written experiment plan is money set on fire. The discipline this syllabus demands is: **write the experiment offline, execute it in a tight window, analyse the traces offline.** Structure every rental as a scripted run, not an exploratory session.

3. Most of the theoretical limit is already embodied in libraries, and the residual gap is elsewhere. The distance between a naïve vLLM deployment and Makora's numbers is *not* mostly hand-written kernels. It is scheduler policy, expert placement, speculative decoding acceptance rates, communication/computation overlap, CUDA graph coverage, and prefill/decode disaggregation ratios. Writing an SM100 GEMM from scratch is an excellent way to *understand* the machine and a poor way to *beat* CUTLASS. Learn to write kernels so you can read, diagnose, and patch them — treat originating them as the exception.

4. Your stated goal and your employer's thesis are in tension, and you should be deliberate about it. RunAnywhere is counter-positioned against NVIDIA — non-GPU, on-device, edge. This syllabus points the other way. That can be entirely rational (the skill is transferable, the market is enormous, and understanding the GPU incumbent sharpens edge positioning), but it should be a choice you've made explicitly rather than a drift you notice in six months.

One correction on the premise: the earlier analysis established 4×B200 as the genuine floor for GLM-5.2 in NVFP4 (8× once you actually exercise long context, KV-bound not weight-bound), and noted Kimi K3 at ~1.4–1.56 TB of quantised weights exceeds a single 8×H200 node. The B300 conclusion for K3 follows from that arithmetic but wasn't stated as such. Verify it yourself in Module 16 — the capacity math is the first thing you should be able to do unaided.

PART I — FOUNDATIONS

Module 0: The arithmetic of inference

Before any hardware. If you cannot do these on a whiteboard, everything downstream is cargo cult.

0.1 Roofline reasoning

- Arithmetic intensity (FLOP/byte); the ridge point; why it moves with precision
- Memory-bound vs compute-bound regimes; where prefill and decode sit and why
- Model Bandwidth Utilisation (MBU) vs Model FLOPs Utilisation (MFU); when each is the right denominator
- Why “GPU utilisation” from `nvidia-smi` is a near-useless metric

0.2 Per-token cost model

- Bytes moved per decode step: dense vs sparse MoE; active-expert streaming
- FLOPs per token, prefill vs decode, attention vs FFN split
- KV cache size: exact formula for MHA, GQA, MQA, and MLA; per-token bytes at FP16/FP8/INT4
- The batch-size crossover: at what B does a decode step become compute-bound
- Weight-streaming ceiling: $\text{decode_tok/s} \leq \text{HBM_BW} / \text{active_bytes_per_token}$

0.3 Latency decomposition

- TTFT = queue + scheduling + prefill compute + first-token overhead
- ITL/TPOT = per-layer HBM term + collective latency term + launch overhead term
- Why the collective term dominates at small batch and the HBM term at large batch
- Serial depth: layer count $\times$ per-layer latency floor; why 92-layer models have a hard interactivity ceiling

0.4 Serving-level queueing

- Little’s Law applied to concurrency, throughput, and latency
- Goodput vs throughput under SLO constraints
- The tok/s/user $\leftrightarrow$ tok/s/GPU Pareto frontier; why a single number is always a lie

- Head-of-line blocking; the prefill/decode interference problem

Deliverable: A spreadsheet or Python model that takes (params, active params, layers, hidden dim, KV config, precision, GPU count, HBM BW, NVLink BW) and emits decode roofline, KV capacity, max concurrency, and the ITL breakdown. Validate it against published GLM-5.2 provider numbers. You will use this tool for the rest of the syllabus.

Primary sources: NVIDIA “LLM Inference Sizing” guidance; the original roofline paper (Williams et al.); vLLM performance docs.

Module 1: Hopper (SM90) — the architecture you skipped

You cannot skip to Blackwell. Every SM100 idiom is a mutation of an SM90 idiom, and every piece of reference code (CUTLASS, FlashAttention, FlashInfer) carries both paths side by side.

1.1 The asynchronous execution model

- Why Hopper broke from the “load-compute-store per thread” model
- `mbarrier`: phase bits, arrive/wait, expect-tx; the transaction-count barrier
- Async pipelines: multi-stage circular buffers in shared memory
- `cp.async` (Ampere) vs bulk async copy (Hopper) — what actually changed

1.2 Tensor Memory Accelerator (TMA)

- Tensor maps (`CUTensorMap`): the descriptor built on host, consumed on device
- `cp.async.bulk.tensor` for global→shared, shared→global
- Multicast: one TMA load feeding an entire cluster
- Swizzle modes (32B/64B/128B) and why they exist — shared-memory bank conflicts at tensor-core width
- im2col mode; box dimensions; out-of-bounds handling
- TMA store and the `cp.async.bulk.commit_group`/`wait_group` protocol

1.3 Warpgroup MMA (`wgmma`)

- The warpgroup as the unit of issue (128 threads); why it isn’t a warp
- Operand sourcing: SMEM descriptors vs registers; accumulator layout in registers

- `wgmma.fence`, `wgmma.commit_group`, `wgmma.wait_group` — the dependency protocol
- Supported shapes and the M64NxK16 family
- FP8 on Hopper: E4M3 vs E5M2, per-tensor scaling, the accumulate-in-FP32 requirement

1.4 Thread block clusters and distributed shared memory

- CGA (cooperative grid array) semantics; cluster launch dimensions
- DSMEM: addressing another CTA's shared memory; `mapa` instruction
- Cluster-wide barriers; when cluster sync beats grid sync
- Practical cluster sizing (2, 4, 8, 16) and occupancy consequences

1.5 Warp specialisation

- Producer/consumer decomposition: DMA warps vs MMA warps vs epilogue warps
- `setmaxnreg.inc` / `setmaxnreg.dec`: dynamic register reallocation between warpgroups
- Why specialisation beats uniform-role kernels on Hopper (latency hiding without occupancy)
- Ping-pong scheduling: two consumer warpgroups alternating on MMA and epilogue

1.6 Persistent kernels

- Grid-sized-to-hardware launches; tile schedulers inside the kernel
- Why persistence kills launch overhead and enables software pipelining across tiles
- Stream-K and split-K decomposition for skinny GEMMs

Deliverable: Write, from scratch, a warp-specialised FP16 GEMM using TMA + `wgmma` + mbarrier pipelining on an H100. Get it to $\geq 70\%$ of cuBLAS on a large square case. Rent one H100 for a weekend — it is materially cheaper than a B200 and the concepts are identical.

Primary sources: PTX ISA §9 (async copy, mbarrier, wgmma); CUDA Programming Guide (clusters, TMA); CUTLASS 3.x `examples/48_hopper_warp_specialized_gemm`; the ThunderKittens and Colfax Hopper GEMM writeups.

PART II — BLACKWELL

Module 2: SM100 / SM103 architecture

2.1 The chip

- Dual-die construction: two reticle-limited dies, NV-HBI interconnect (~ 10 TB/s), presented as one CUDA device
- What “one device, two dies” means for L2 (it isn’t unified in the way you’d assume) and for scheduling locality
- B200: 192 GB HBM3e, ~ 8 TB/s; SM count, clock behaviour, TDP
- B300 / Blackwell Ultra (SM103): 288 GB HBM3e, increased dense FP4 throughput, substantially reduced FP64, faster SFU transcendental rate (directly relevant to attention softmax)
- **The SM100 vs SM120 trap:** consumer/workstation Blackwell (RTX PRO 6000, GB202) is SM120 and has an entirely different tensor-core programming model — no `tcgen05`, no TMEM. Code written for one does not run on the other. Know which arch every repo targets before you clone it.
- `sm_100a` vs `sm_100f` vs `sm_100` compilation targets; architecture-accelerated features and forward-compat implications

2.2 Tensor Memory (TMEM)

- The new address space: ~ 256 KB per SM, organised as 128 lanes $\times$ 512 columns $\times$ 32 bit
- Why NVIDIA moved accumulators out of registers: register file pressure was the binding constraint on Hopper
- `tcgen05.alloc` / `tcgen05.dealloc`: allocation granularity, the column-count constraint, why allocation is a warp-level collective
- `tcgen05.ld` / `tcgen05.st`: the shapes (`16x64b`, `16x128b`, `16x256b`, `32x32b`), the `.pack` / `.unpack` modifiers, lane-to-thread mapping
- `tcgen05.cp`: SMEM $\rightarrow$ TMEM copies for operand staging
- TMEM as a scarce resource: how allocation size caps the tile size caps the occupancy

2.3 Fifth-generation tensor cores: `tcgen05.mma`

- **Single-thread issue.** One elected thread (`elect.sync`) launches the MMA for the whole CTA. This is the deepest break from everything you know.
- Instruction descriptors: the 32-bit `idesc` encoding shapes, transposes, scale modes, sparsity
- SMEM descriptors for A and B operands: base address, leading/stride byte offsets, swizzle mode
- Accumulate-or-overwrite flag; the `.commit` / `mbarrier` completion protocol
- Supported kinds: `.kind::f16`, `.kind::tf32`, `.kind::f8f6f4`, `.kind::mxf8f6f4`, `.kind::mxf4`, `.kind::mxf4nvf4`
- **CTA-pair (2-SM) MMA:** two SMs in a cluster cooperating on one MMA, operands split across the pair, `.cta_group::2` semantics. This is where peak FLOPs live and where most hand-written kernels fail.
- Sparsity support (2:4 structured) and its practical irrelevance for LLM inference today

2.4 Block-scaled MMA

- The microscaling concept: a shared exponent per small block of elements
- NVFP4: E2M1 elements, block size 16, **FP8 (E4M3) scale factors**, plus a per-tensor FP32 global scale
- MXFP4: E2M1 elements, block size 32, **E8M0 (power-of-two) scale factors**
- MXFP8, MXFP6 and the `f8f6f4` mixed-input path
- **Scale-factor layout in TMEM** — the interleaved SFA/SFB format. This is the single most error-prone detail in Blackwell kernel work; budget real time for it.
- Effective bytes-per-weight arithmetic: why NVFP4 is ~ 0.5625 B/param, not 0.5

2.5 Scheduling and launch

- Cluster Launch Control (CLC): dynamic work distribution for persistent kernels
- Tile scheduler design on SM100; rasterisation order and L2 locality
- Grid constant / `__grid_constant__` and TMA descriptor passing
- Programmatic Dependent Launch (PDL) — overlapping the tail of one kernel with the prologue of the next

2.6 Memory hierarchy and limits

- Registers $\rightarrow$ TMEM $\rightarrow$ SMEM $\rightarrow$ L2 $\rightarrow$ HBM: capacity, bandwidth, and latency at each level

- Shared memory capacity per SM and the dynamic SMEM opt-in
- L2 residency control, `cudaAccessPolicyWindow`, persisting L2 for weights that fit
- Distributed shared memory across clusters on SM100

2.7 Power, clocks and thermals

- Why sustained FP4 GEMMs clock lower than the marketing peak
- DVFS behaviour on liquid- vs air-cooled B200; how this invalidates naïve benchmark comparisons
- `nvidia-smi -q -d CLOCK,PERFORMANCE`, clock-limit reason codes, and locking clocks for reproducible measurement

Deliverable: Port your Module 1 Hopper GEMM to SM100 using `tcgen05.mma` with TMEM accumulators, then extend it to a CTA-pair variant, then to NVFP4 with block scales. Compare each against cuBLAS/CUTLASS. Document exactly where your version loses.

Primary sources: PTX ISA §9.7.16 (`tcgen05`); CUDA Programming Guide (Blackwell section); CUTLASS 4.x `examples/70_blackwell_*` series and the CuTe DSL docs; NVIDIA Blackwell architecture whitepaper; OCP Microscaling Formats specification.

Module 3: Numerics and quantisation

3.1 Format fundamentals

- IEEE FP16/BF16/FP32/TF32; dynamic range vs precision trade
- FP8: E4M3 (weights/activations) vs E5M2 (gradients); saturation vs overflow behaviour
- FP6, FP4 (E2M1) value sets; why FP4 has 16 representable values and what that means for outliers
- Block scaling as a dynamic-range recovery mechanism; block size as an accuracy/overhead dial

3.2 Quantisation algorithms

- Post-training quantisation: round-to-nearest baseline and why it fails on LLMs
- Outlier-driven failure: activation outliers, the emergent-feature phenomenon

- SmoothQuant: migrating difficulty from activations to weights
- AWQ: activation-aware salient-channel protection
- GPTQ / OBT lineage: second-order error compensation
- Rotation methods: QuaRot, SpinQuant, Hadamard transforms; why rotation flattens outliers and what it costs at runtime
- Quantisation-aware training and NVFP4 QAT recipes; when it's worth the compute

3.3 KV cache quantisation

- FP8 KV: the default, near-free
- INT4/FP4 KV: per-head and per-token scaling schemes
- Accuracy degradation as a function of context length — why perplexity hides this and needle-in-haystack exposes it
- Dequant cost inside the attention kernel; fused dequant-in-register patterns

3.4 Evaluation discipline

- Why perplexity is insufficient and near-useless as a quantisation gate
- Task-level evals: coding, agentic tool-calling, long-context retrieval
- The specific failure mode of quantised MoE: router perturbation changing expert selection
- Constructing a regression harness you trust before you touch a quantisation knob

Deliverable: Quantise a mid-size MoE (30–40B class) to NVFP4 yourself using TensorRT Model Optimizer or LLM Compressor. Produce an accuracy delta table across four eval categories. Then break it deliberately — pick a bad block size or skip rotation — and show which eval catches it first.

Primary sources: OCP MX spec; SmoothQuant, AWQ, GPTQ, QuaRot papers; NVIDIA TensorRT Model Optimizer docs; NVIDIA's NVFP4 accuracy blog series.

PART III — KERNELS

Module 4: GEMM engineering

4.1 CUTLASS / CuTe

- `Layout` algebra: shapes, strides, composition, complement, division, product
- `Tensor`, `TiledCopy`, `TiledMMA`; the atom $\rightarrow$ tiled-atom hierarchy
- The CuTe Python DSL (CUTLASS 4.x) as the fast path for Blackwell experimentation
- Collective builders: `CollectiveMainloop`, `CollectiveEpilogue`, kernel schedules
- Reading a CUTLASS kernel schedule name and knowing exactly what it does

4.2 Blackwell GEMM schedules

- Warp-specialised mainloop with TMEM accumulation
- 1-SM vs 2-SM (CTA-pair) schedules; when the pair wins
- Tile shape selection under TMEM capacity constraints
- Epilogue fusion: bias, activation, scaling, quantise-on-store, split-D
- Stream-K on Blackwell for irregular problem shapes

4.3 Low-precision GEMM specifics

- Block-scaled GEMM plumbing: where scale tensors live, how they're staged
- Mixed-input GEMM (FP4 weights $\times$ FP8 activations, etc.)
- The dequantise-fuse decision: in-kernel vs pre-pass
- DeepGEMM's design: JIT compilation per shape, why it beats a static library for MoE, and the 10+ minute cold-start cost it imposes in production

4.4 Grouped GEMM for MoE

- Variable-M grouped GEMM: the problem statement
- Scheduling strategies: per-group tiles vs a global tile queue
- Masked/contiguous layouts for token-to-expert grouping
- Why load imbalance across experts is a *kernel* problem before it's a *systems* problem

Deliverable: Benchmark suite comparing your kernel, CUTLASS, DeepGEMM, and cuBLAS across the exact GEMM shapes GLM-5.2 decode actually issues (extract them by tracing a real forward pass). Produce a shape-vs-best-implementation map.

Primary sources: CUTLASS repo ([media/docs](#)), CuTe tutorials); DeepGEMM source; Colfax Research articles on Blackwell GEMM.

Module 5: Attention kernels

5.1 The FlashAttention lineage

- FA-1: tiling + online softmax, IO-awareness
- FA-2: work partitioning, reduced non-matmul FLOPs, better parallelism over sequence
- FA-3 (Hopper): warp specialisation, TMA, FP8 attention, the ping-pong and interleaved schedules, softmax/GEMM overlap
- FA-4 (Blackwell): [tcgen05](#) integration, revised online-softmax formulation, faster exponential approximation, exploiting the improved SFU rate
- Reading these as a *sequence of hardware adaptations*, not four separate algorithms

5.2 Decode-time attention

- Why decode attention is a GEMV-shaped, bandwidth-bound problem
- FlashDecoding / split-KV: parallelising over the KV dimension and the second-pass reduction
- Paged attention: block tables, non-contiguous KV, the indirection cost
- Kernel selection by (batch, seqlen, head config) — there is no single best decode kernel

5.3 Attention variants

- MHA → MQA → GQA: the KV-head reduction and its TP consequences
- **MLA (Multi-head Latent Attention):** latent KV compression, weight absorption in decode, why MLA decode is compute-heavy but KV-light
- Why MLA breaks naïve tensor parallelism (effectively one KV head) and forces data-parallel attention

- Sparse attention: block-sparse, native sparse attention, and sparse-MLA decode kernels; the top-k selection kernel as a first-class cost
- Sliding window, attention sinks, hybrid full/local layer stacks

5.4 Long context

- Ring attention and context parallelism
- Chunked prefill and its interaction with attention kernel efficiency
- KV compression, eviction (H2O, SnapKV) and quantisation — and why each is dangerous for agentic workloads
- The million-token regime: what actually dominates (it is not FLOPs)

5.5 FlashInfer

- Architecture: attention as composable templates, JIT specialisation
- The plan/run split and why it exists
- Block-sparse abstraction unifying paged, ragged, and sparse layouts
- How vLLM/SGLang/TRT-LLM consume it

Deliverable: Implement a paged FP8 decode-attention kernel for GQA at 128K context. Then profile it against FlashInfer and explain every percentage point of the gap using NCU counters.

Primary sources: FlashAttention 2/3/4 papers; FlashInfer paper and repo; DeepSeek V3/V3.2 technical reports (MLA and sparse attention).

Module 6: MoE execution

6.1 The MoE forward pass, mechanically

- Router: gating GEMM, top-k selection, softmax/sigmoid normalisation, auxiliary-loss-free bias correction
- Permutation: token sorting/scatter by expert, the index-building kernel
- Grouped GEMM over experts (up-proj, gate, down-proj)
- Un-permutation and weighted combine
- Shared/always-on experts and how they change the schedule

6.2 Where MoE decode actually loses time

- Expert imbalance at low batch: a “sparse” model behaving dense
- Kernel launch overhead across many small per-expert GEMMs
- Weight streaming: active-expert bytes as the true bandwidth denominator
- Router non-determinism under quantisation

6.3 Fusion strategies

- Fused MoE kernels: routing + permute + GEMM in one launch
- Persistent MoE kernels; TileRT-style whole-graph persistence and why it produced outsized wins on single-node decode
- CUDA graph capture across the MoE block — the dynamic-shape obstacle and how implementations dodge it

Deliverable: Instrument a real MoE decode step at batch 1, 8, 64, 256 and produce a per-phase time breakdown (router / permute / GEMM / combine / collective). Identify the crossover batch where imbalance stops mattering.

Primary sources: DeepSeek-V3 technical report; Mixtral paper; vLLM and SGLang fused-MoE kernel sources; DeepGEMM.

Module 7: The remaining kernel surface

7.1 Elementwise and normalisation

- RMSNorm/LayerNorm fused with residual add and output quantisation
- RoPE variants (interleaved vs half-split, YaRN/NTK scaling) and fusing into attention prologue
- SwiGLU/GeGLU fusion; the gate-up fused weight layout

7.2 Sampling and output

- On-GPU top-k / top-p / min-p; sorting vs partial-selection algorithms
- Repetition and presence penalties at batch scale
- Structured decoding: grammar/JSON-constrained masking, the mask-computation cost, why it can dominate small-batch decode

- Logit processors and the CPU-GPU sync trap

7.3 Launch overhead elimination

- CUDA Graphs: capture, instantiation, replay; graph update for changing pointers
- Piecewise graph capture around dynamic-shape regions (the vLLM v1 approach)
- Multiple graph variants per batch-size bucket; padding policy
- Measuring launch overhead honestly (it is invisible without a timeline profiler)

Deliverable: Take a working decode loop and cut per-step CPU overhead to under 200 μ s. Prove it with an nsys timeline showing zero gaps between kernels.

Module 8: Measurement and profiling

8.1 Nsight Systems

- Timeline reading: CUDA API vs kernel vs NCCL vs CPU rows
- NVTX instrumentation of your own serving stack
- Finding gaps: launch-bound, sync-bound, comm-bound signatures
- Multi-GPU and multi-node trace collection and alignment

8.2 Nsight Compute

- Section sets; speed-of-light, memory workload analysis, warp state sampling
- SM100-specific metrics: tensor-core utilisation, TMEM traffic, TMA throughput
- Roofline chart in NCU and how to read it correctly
- Source-level counters and SASS correlation
- PM sampling on Blackwell for range-based analysis
- Why profiling a persistent kernel requires a different methodology

8.3 SASS and compiler behaviour

- `cuobjdump -sass`, `nvdiasm`; reading Blackwell SASS well enough to spot register spills and predication
- `-Xptxas -v`, register/SMEM usage reporting, occupancy calculation by hand
- When to write inline PTX and when the compiler is already doing better

8.4 System-level observability

- DCGM metrics, per-GPU power/clock/thermal telemetry
- `nvidia-smi nvlink -gt d`, NVLink error counters, link flapping detection
- NCCL debug output, topology dumps, algorithm/protocol selection logs

Deliverable: A reproducible profiling harness for your serving stack that produces, from one command, a per-layer time breakdown with MBU per layer and a flagged list of the top five recoverable inefficiencies.

PART IV — SCALING OUT

Module 9: Single-node multi-GPU

9.1 Topology

- NVLink 5: per-link bandwidth, link count, 1.8 TB/s aggregate per GPU
- NVSwitch within an 8-GPU baseboard; all-to-all bandwidth guarantees
- PCIe topology, NUMA affinity, CPU-GPU binding; why `numactl` matters
- GB200/GB300 superchip: Grace CPU, NVLink-C2C at 900 GB/s, coherent host memory as an offload target
- Reading `nvidia-smi topo -m` and knowing what it implies for your parallelism plan

9.2 Collectives

- The primitives: all-reduce, all-gather, reduce-scatter, all-to-all, broadcast
- NCCL algorithms: ring, tree, NVLS (with in-network reduction), CollNet
- Protocols: Simple, LL, LL128 — the latency/bandwidth trade at small message sizes
- Latency vs size curves: the flat region that dominates decode
- One-shot vs two-shot all-reduce for small tensors
- NCCL environment tuning: `NCCL_ALGO`, `NCCL_PROTO`, `NCCL_NTHREADS`, buffer sizes, and why defaults are wrong for inference

9.3 Beyond NCCL

- NVSHMEM: one-sided PGAS communication, why MoE all-to-all wants it
- Symmetric memory and PyTorch's `symm_mem` primitives
- Custom all-reduce implementations in vLLM/TRT-LLM for small-message latency
- Multicast/multimem PTX instructions for NVLS-accelerated reductions

9.4 Overlap

- Communication/computation overlap: the fundamental technique
- Fused all-reduce + RMSNorm + quantise epilogues
- Async tensor parallelism (Flux-style): decomposing GEMM tiles to overlap with comm
- Stream priorities and separate comm streams; the deadlock hazards
- Microbatch pipelining to hide all-to-all in MoE

Deliverable: Measure the actual all-reduce latency curve on an $8 \times B200$ node across message sizes from 4 KB to 1 GB, for every algorithm/protocol combination. Overlay your decode step's actual message sizes. Then predict, and verify, the TP-degree that minimises ITL.

Module 10: Multi-node and rack scale

10.1 NVL72 and rack architecture

- 72 GPUs in a single NVLink domain; what changes when NVLink is no longer node-local
- Aggregate bandwidth, switch tiers, failure domains
- Why rack-scale improves aggregate throughput and *worsens* single-stream interactivity — the per-layer HBM term shrinks while collective latency does not
- Power, cooling and the practical realities of density

10.2 Scale-out networking

- InfiniBand NDR/XDR vs RoCEv2; when each is chosen
- Rail-optimised topology; why GPU i on every node shares a rail
- GPUDirect RDMA; the path a byte takes from GPU HBM to a remote GPU's HBM
- IBGDA (GPU-initiated communication) and why it matters for MoE dispatch

- Congestion control, adaptive routing, and the tail-latency consequences

10.3 Multi-node collectives

- Hierarchical algorithms: intra-node NVLink phase + inter-node network phase
- SHARP in-network reduction
- Bandwidth cliff at the node boundary: quantify it before designing parallelism
- Fault handling: NCCL timeouts, hung-collective diagnosis, straggler detection

Deliverable: Design (on paper, defensibly) a Kimi K3 deployment across an NVL72 slice: parallelism assignment, expert placement, KV budget, expected ITL, and the three most likely failure modes. This is the artifact you'd show a serious infrastructure team.

Module 11: Parallelism strategy

11.1 The dimensions

- Tensor parallelism: column/row splits, where all-reduces land, TP within vs across nodes
- Pipeline parallelism: micro-batching, 1F1B, interleaved schedules, bubble arithmetic; why PP is more attractive for inference than training
- Expert parallelism: expert placement, all-to-all dispatch/combine, capacity factors
- Data parallelism and **attention-DP** — the standard answer for MLA models
- Sequence/context parallelism for long-context prefill
- Hybrid layouts: TP inside a node, EP across nodes, DP for attention

11.2 Choosing

- Decision procedure: memory capacity first, then collective cost, then load balance
- Why TP degree beyond 8 usually loses on decode
- The all-to-all cost model for EP; when wide-EP (64–320 GPUs) pays and when it doesn't
- Prefill and decode want *different* parallelism — the strongest argument for disaggregation

11.3 Expert load balancing

- Measuring expert skew in production traffic
- Redundant/replicated hot experts
- EPLB-style rebalancing; static placement from offline traces vs dynamic migration
- The load-imbalance tax expressed as effective bandwidth loss

Deliverable: For GLM-5.2 on $8 \times B200$, enumerate every viable parallelism configuration, predict ITL and throughput for each with your Module 0 model, then rank them. Later, measure and score your own predictions.

PART V — THE SERVING SYSTEM

Module 12: Serving architecture

12.1 Batching and scheduling

- Continuous/in-flight batching: the core mechanism
- Chunked prefill: mixing prefill chunks into decode batches; chunk-size tuning
- Prefill/decode interference and the ITL spikes it causes
- Scheduler policies: FCFS, priority, SLO-aware admission control, preemption and swap-out
- Batch-size bucketing for CUDA graph reuse

12.2 KV cache management

- Paged allocation: block size selection, fragmentation, copy-on-write for forks
- Prefix caching: radix/trie structures, eviction policy, hit-rate measurement
- Hierarchical cache: HBM $\rightarrow$ host RAM (over C2C or PCIe) $\rightarrow$ NVMe $\rightarrow$ object store
- KV offload bandwidth arithmetic: when offload beats recompute
- Cache-aware routing: sending a request to the replica that already holds its prefix
- Your existing spawn-tree/agent KV work maps directly onto this module — connect them explicitly

12.3 Multi-tenancy

- LoRA/adaptor serving at scale; batched heterogeneous adapters

- Per-tenant quotas, fairness, and priority classes
 - Cold start: weight loading, JIT warmup, graph capture time; the real cost of a scale-up event
-

Module 13: Speculative decoding

13.1 Mechanics

- Draft-then-verify: the rejection sampling argument for output-distribution preservation
- Acceptance length as the single governing metric
- The throughput trade: speculation adds compute to save memory traffic — it *hurts* at high batch

13.2 Methods

- Independent draft models; the size/quality trade
- Medusa: multiple decoding heads
- EAGLE-1/2/3: feature-level autoregression, dynamic draft trees
- MTP (multi-token prediction) heads trained with the base model — GLM-5.2 and DeepSeek both ship these
- N-gram/prompt-lookup speculation for repetitive agentic workloads

13.3 Engineering

- Tree attention and the verification kernel
- Speculation depth/width tuning against acceptance rate
- Dynamic speculation: turning it off as batch size grows
- Measuring acceptance on *your* traffic, not on the paper's benchmark

Deliverable: Measure acceptance length for an MTP-equipped model on your real workload distribution. Compute the batch size at which speculation becomes net-negative. Implement the switch.

Module 14: Disaggregated serving

14.1 The architecture

- Why prefill (compute-bound, latency-tolerant, large-batch-friendly) and decode (memory-bound, latency-critical) belong on different hardware pools
- P/D ratio derivation from ISL/OSL distribution
- Independent scaling and independent parallelism per pool

14.2 KV transfer

- Transfer mechanisms: NVLink, RDMA, NIXL abstraction layers
- Layer-wise streaming to overlap transfer with prefill
- Transfer-time budget vs recompute-time budget
- Mooncake and LMCache designs

14.3 Orchestration

- NVIDIA Dynamo: router, planner, KV block manager
- Global KV-aware routing across a cluster
- SLO-driven autoscaling of P and D pools independently
- Failure handling mid-transfer

Deliverable: Stand up a two-pool disaggregated deployment (even at small scale on 2 GPUs) and produce the TTFT/ITL comparison against a co-located baseline on identical traffic.

Module 15: The framework stack

Learn these as *codebases you can modify*, not as products you configure.

15.1 Inference engines

- **TensorRT-LLM:** the PyTorch backend, custom ops, TRT-LLM Gen kernels, AutoDeploy, the plugin surface
- **vLLM v1:** scheduler, the executor/worker split, attention backend abstraction, piecewise CUDA graphs, torch.compile integration

- **SGLang**: RadixAttention, the two-batch overlap scheduler, DeepEP integration, its MoE path
- Reading engine source to answer “why is my kernel not being selected”

15.2 Kernel libraries

- FlashInfer, CUTLASS/CuTe DSL, DeepGEMM, DeepEP, Triton (and Triton’s Blackwell limitations — where it cannot yet reach `tcgen05` peak)
- How to inject a custom kernel into each engine without forking it permanently

15.3 Orchestration

- Dynamo, KServe/production stacks, Kubernetes GPU scheduling, MIG (and why you won’t use it here)
- Model weight distribution and load-time optimisation

Deliverable: Land one non-trivial upstream-quality patch in vLLM or SGLang. This is also the highest-leverage public-reputation move available to you, and it costs a fraction of what kernel research costs.

Module 16: Model-specific bring-up

16.1 GLM-5.2

- Architecture inventory: ~744–753B total, ~40B active, 92 layers, MoE config, attention variant, MTP heads
- NVFP4 weight footprint (~420–460 GB) → minimum GPU count → the $4\times$ vs $8\times$ decision driven by KV, not weights
- Parallelism plan, expert placement, speculation configuration
- The 92-layer serial-depth ceiling on interactivity

16.2 Kimi K3

- ~2.8T parameters, ~1.4–1.56 TB quantised weights, 1M context window
- Why capacity forces B300 (288 GB) or rack-scale, and what that does to per-user latency
- KV budget at 1M context: the dominant term at any real concurrency

- Wide-EP as the only economically coherent deployment shape
- Licence constraints (revenue/attribution thresholds) — a real deployment gate, not a footnote

16.3 Generalised bring-up procedure

- Reading a new model's config and deriving the full cost model in under an hour
- Identifying unsupported ops and the minimum kernel work to close them
- Numerical validation against the reference implementation, layer by layer
- Building the eval gate before the performance work, not after

Deliverable: A one-page capacity-and-parallelism spec for each model, derived independently with your Module 0 tool, with every assumption stated and every number traceable.

Module 17: Benchmarking and SLO engineering

17.1 Methodology

- ISL/OSL distributions matched to real workloads (agentic coding traffic looks nothing like ShareGPT)
- Concurrency sweeps; the tok/s/user vs tok/s/GPU Pareto curve as the only honest output
- Warmup, clock locking, run-to-run variance, statistical significance
- The benchmarking sins: single-concurrency numbers, unstated ISL, unlocked clocks, cherry-picked percentiles

17.2 Tooling

- genai-perf, vLLM's benchmark suite, SGLang bench, custom load generators
- Trace replay from production logs
- Continuous performance regression testing in CI

17.3 Economics

- Cost per million tokens; GPU-hour amortisation; duty-cycle sensitivity
- Break-even analysis vs hosted APIs

- The uncomfortable finding you already derived: self-hosting loses below $\sim 20\text{--}40$ B tokens/month at high duty cycle

Deliverable: Publish one rigorous, reproducible benchmark with full methodology. The field is starved of these. This is a reputation asset disproportionate to its cost.

Module 18: Operations

- XID error taxonomy; correlating GPU faults to workload
 - ECC, row remapping, and when to drain a node
 - NVLink error counters and degradation detection before failure
 - Thermal/power throttle detection in production telemetry
 - Checkpoint/restore, graceful drain, rolling upgrades of a stateful KV-holding service
 - Capacity planning and headroom policy under bursty agentic traffic
-

Sequencing

Do not attempt these in parallel. A realistic ordering, assuming heavy weekly hours:

Phase	Modules	Focus	Hardware needed
1	0	The arithmetic	None
2	1	Hopper foundations	1 × H100, weekends
3	2, 3	Blackwell + numerics	1 × B200, scripted runs
4	4, 5, 6, 7, 8	Kernels + profiling	1 × B200
5	12, 13, 15	Serving systems	1–2 × any GPU
6	9, 11	Single-node scaling	8 × B200, tight windows
7	10, 14, 16, 17, 18	Rack scale + production	Contract access

Reorder if you want ROI sooner: Phase 5 before Phase 4. Serving-system work (scheduling, prefix caching, speculation tuning, graph coverage) yields larger measured gains per engineering hour than kernel work does, needs far cheaper hardware, and is where most of the deployed-performance gap actually lives. Kernel mastery is the deeper skill; systems mastery is the faster payoff. Given that you have no B200 access secured, taking Phase 5 first is probably the correct sequencing rather than merely the convenient one.

What to skip: FP64 anything, structured sparsity, training-specific parallelism (ZeRO, FSDP internals, gradient compression), MIG, and graphics/CUDA-interop. None of it serves this goal.

The tutor prompt

Use this per subtopic. Fill the three bracketed slots.

You are a principal-level GPU systems engineer who has shipped production inference kernels on Hopper and Blackwell and has debugged multi-node NVLink deployments in anger. I am learning to serve trillion-parameter sparse MoE models on B200/B300 nodes at as close to the hardware limit as achievable.

MY BACKGROUND – calibrate to this exactly, do not over- or under-explain:

- Fluent in CUDA C++ on consumer Ampere (SM86): threads, warps, shared memory, coalescing, occupancy, basic tiled GEMM. All of it single-GPU, all of it on 4 GB RTX 3050.
- ZERO hands-on with SM90 or SM100: no TMA, no wmma, no tcgen05, no TMEM, no thread block clusters, no warp specialisation, no persistent kernels.
- ZERO hands-on with multi-GPU: no NCCL, no tensor/pipeline/expert parallelism, no RDMA, no multi-node anything.
- Strong ML systems context: quantisation, inference engines, roofline reasoning, KV cache architecture. I build inference runtimes professionally, just not on datacentre GPUs.

TOPIC: [TOPIC]

DEPTH: [pick one – CONCEPTUAL / IMPLEMENTATION / ISA-LEVEL]

HOW TO ANSWER:

1. Start with the hardware constraint that forced this design to exist. Not "what it is" – WHY the silicon made it necessary. If it replaced an earlier mechanism, state what that was and what specifically broke.
2. Give the precise mechanism: exact instruction names, operand layouts, memory spaces, synchronisation protocol, and the ordering guarantees. Name the PTX instructions or API calls. If there is a descriptor or bit-field encoding involved, spell out the fields.
3. Give real numbers with units: bytes, cycles, latencies, bandwidths, capacities, throughputs. If a number is architecture-specific, state which architecture. If you do not know a number precisely, say "I don't have a reliable figure for this" – never estimate silently.
4. Show working code. Minimal but COMPLETE and compilable – CUDA C++, PTX, or CuTe DSL as appropriate. Annotate the lines that are non-obvious. If the real-world version differs materially from the minimal version, say how.
5. Contrast against what I already know: how this differs from SM86, and how it differs from SM90. The delta is how I will actually retain this.
6. List the top failure modes: what silently produces wrong numerics, what silently costs 3× performance, what the compiler will not warn you about.
7. Tell me how to VERIFY it on real hardware: the exact ncu metrics or nsys view, what value indicates success, what value indicates the specific failure modes above.
8. Cite primary sources by section: PTX ISA section number, CUDA Programming Guide chapter, specific CUTLASS example, or the paper. So I can go check you.
9. End with: (a) three questions I should be able to answer if I understood this, and (b) one concrete exercise with a stated success criterion.

RULES:

- Assume I want the mechanism, not the intuition. Analogies are permitted only

AFTER the precise version, never instead of it.

- Do not soften, do not pad, do not restate my question back to me.
- If something is genuinely contested, undocumented, or changed between CUDA versions, say so explicitly and tell me which version you are describing.
- If my question contains a false premise, correct it before answering.
- If this topic has a prerequisite I have clearly not covered, tell me to go learn that first instead of answering.

Depth dial:

- **CONCEPTUAL** — mechanism and trade-offs, no code. Use when triaging whether a topic matters to you.
- **IMPLEMENTATION** — working kernel-level code, CUTLASS/CuTe idioms, real API calls. The default.
- **ISA-LEVEL** — PTX/SASS, descriptor bit fields, instruction latencies, register allocation. Use for Modules 2 and 4 only.

Two supplementary prompts worth keeping:

Debug mode:

Here is my kernel and its ncu profile: [PASTE]. Expected X, achieving Y. Do not rewrite it. Diagnose it: rank the three most probable causes by likelihood, state the specific counter or metric that would confirm or eliminate each, and tell me what to measure next. Then, and only then, propose a fix for the top-ranked cause.

Design-review mode:

Here is my deployment plan: [PASTE – parallelism, precision, batching, hardware]. Attack it. Where does the arithmetic not close? What have I assumed that only holds at low concurrency or short context? What breaks first under production traffic, and at what load? Give me the numbers that disprove me if they exist.

Two additions I'd make to your plan

Ship an artifact, not just knowledge. The field has almost no rigorous, reproducible public benchmarks of batch-1 MoE decode with per-layer MBU on Blackwell. Producing

one costs perhaps \$700 of rented single-B200 time and would be a stronger credential than any amount of private study — it demonstrates measurement discipline, which is rarer and more valuable than kernel-writing ability.

Secure hardware access before Module 2, not after. The syllabus is worthless without it. Options in rough order of tractability: NVIDIA Inception or the Developer Program for credits (your company's YC status and NPU partnership work help here), the Qualcomm/NVIDIA partner channels you already have open, neocloud trial credits, academic allocation through SRM, or a written proposal to your team for a scoped budget with a defined deliverable. Do this first. Everything downstream is gated on it.